Storage Designs in Practice

Redis · etcd · the LSM Family

Week 10B — Advanced Operating Systems

Recall from 10A: The Write-Path

Week 10A gave us three questions for durability and recovery:

1. **Where does truth live first?** — RAM, WAL, quorum log, or object service?
2. **When is `OK` honest?** — after local work, after `fsync`, after a majority?
3. **What cleanup follows later?** — snapshot, rewrite, compaction, defrag?

Beyond the Write Path

There are still other important factors:

1. **Read path:** where do reads come from, and how stale can they be?
2. **Failure path:** what happens on leader loss, disk loss, or partition?
3. **Boundary:** what is atomic and ordered — one key, one command, one shard, or a transaction?

Today's Learning Objectives

1. Describe Redis's single-threaded architecture and explain why `appendfsync` is a client-facing contract, not just a config option
2. Trace a write through etcd's Raft → WAL → bbolt path and explain why each step exists
3. Walk through the LSM write path (WAL → MemTable → SSTable → compaction) and identify the three amplification costs
4. Compare Redis, etcd, and LSM engines by write path, read path, failure behavior, cleanup cost, and atomicity/ordering boundary

Road Map

UNIT	SYSTEM FAMILY	CENTRAL QUESTION
A	Redis	What happens when truth lives in RAM?
B	etcd	What changes when correctness is the product?
C	LSM engines	How do you handle data too large for memory?
—	Object storage	A different problem, a different design
—	Synthesis	Core write-path lens, plus read/failure/boundary checks

Unit A

Redis: Truth Lives in RAM

Redis: Design

In 10A you learned Redis's persistence mechanisms (AOF as WAL, `appendfsync`, RDB snapshots) and measured their cost in Lab 10A. Now we ask the design question: **why did Redis make these choices?**

Redis keeps **all hot state in memory** and treats durability as an opt-in cost. It is a data structure server — it exposes hash tables, sorted sets, lists, and streams directly over the network. Persistence is a secondary feature, not the core identity.

When it matters: Caches, session stores, rate limiters, real-time leaderboards.

> Think of Redis as the fastest possible answer design point. Everything else follows from that one choice.

Redis: Why a Single-Threaded Event Loop?

Redis processes commands on **one event-loop thread** using `epoll` / `kqueue`.

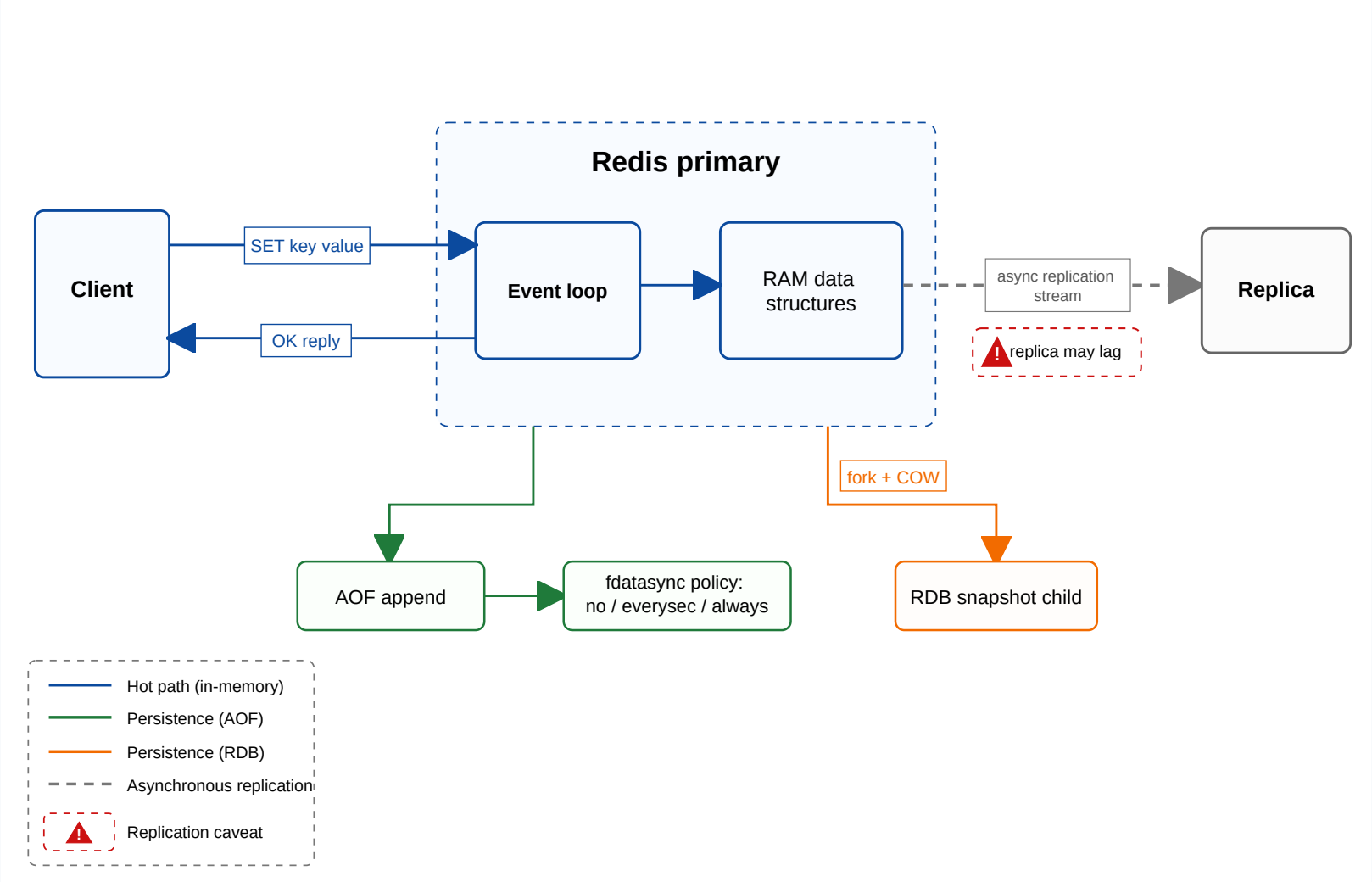
Why this is enough

- Each command is a tiny data-structure op (SET, GET, ZADD, LPUSH) — typically **< 1 µs**
- The bottleneck is the network, not the CPU
- No lock contention; every command is atomic by construction
- I/O-heavy background work (RDB/AOF rewrite) uses **child processes** via `fork()`

Threads vs. event loop

- Thread per connection: simple to write, but locks + context switches dominate when each unit of work is < 1 µs
- Event loop: one thread, thousands of connections via non-blocking I/O — wins for short, CPU-cheap requests

Redis: Architecture Overview



Redis in the Wild

DEPLOYMENT	USE CASE
Twitter / X	Timeline cache: precomputed home timelines per user
GitHub	Session store, rate limiting, background job queue (Resque)
Stack Overflow	Hot question/answer cache; multi-tier read cache
Discord	Presence service, real-time fanout counters
Pinterest	Feed assembly cache, ML feature store
AWS ElastiCache · Azure · GCP	Managed Redis-as-a-service used by thousands of apps

> Common pattern: Redis sits **in front of** a slower system of record (MySQL, S3, an LSM KV) to absorb hot reads and short-lived writes. Truth still lives in the slower store; Redis carries the speed.

Redis: Reads, Replicas, and Failover

Redis replication is usually **primary → replica** and asynchronous.

OPERATION	USUAL PATH	CONSEQUENCE
Write	client → primary memory → `OK`	fastest path; durability depends on config
Read	primary, or replica if allowed	replica reads may be stale
Failover	Cluster promotes a replica	any unreplicated write is gone

Redis does not run a quorum protocol before replying.

Redis Persistence: RDB vs. AOF

Two mechanisms, two different durability stories:

	RDB SNAPSHOT	AOF (APPEND-ONLY FILE)
What it keeps	Periodic full memory image	Append-only log of write commands
Good at	Compact files, fast restart	Smaller crash-loss window
Gives up	All writes since last snapshot	Ongoing I/O cost + rewrite overhead

Simple framing:

- RDB = the cheap restart story (good backups, bad crash recovery)
- AOF = the better durability story (good crash recovery, ongoing cost)

Many production deployments use **both**: AOF for crash recovery, RDB for fast backup/restore.

Recall: AOF and `appendfsync`

You already know:

- The AOF is a text log of Redis commands in RESP format
- `appendfsync` controls when Redis calls `fdatasync` — you counted the calls with `strace`
- The throughput gap between `always` and `no` is the cost of per-write `fdatasync`

But more questions:

- Why does Redis choose this design over alternatives?
- What happens when the AOF grows too large?
- What does Redis give up by using async replication?

`appendfsync` Revisited: From Mechanism to Design Decision

You measured the mechanism in Lab 10A. Now frame it as a **design decision**:

POLICY	LAB 10A: WHAT YOU MEASURED	DESIGN IMPLICATION
`always`	Per-write `fsync`, lowest throughput	Safe data storage on persistent devices
`everysec`	Group `fsync`, moderate throughput	We can accept 1s loss of data
`no`	OS writeback, highest throughput	Speed maximized, but `OK` is a lie about durability

The choice exposes a fundamental design trade-off that every storage system must make somewhere.

Redis is unusual in exposing this knob directly to the operator. Most databases (PostgreSQL, etcd) fix this choice for you.

> **The design question:** Why does Redis let you turn off durability? Because Redis's primary identity is a data structure server, not a database. Durability is opt-in because many Redis workloads (caches, sessions) don't need it.

Worked Example: AOF Rewrite via fork → COW → Rename

Problem

The AOF file grows without bound. How does Redis compact it without blocking foreground operations?

Approach

Use `fork()` to snapshot the in-memory state, rewrite in the child, and swap files atomically.

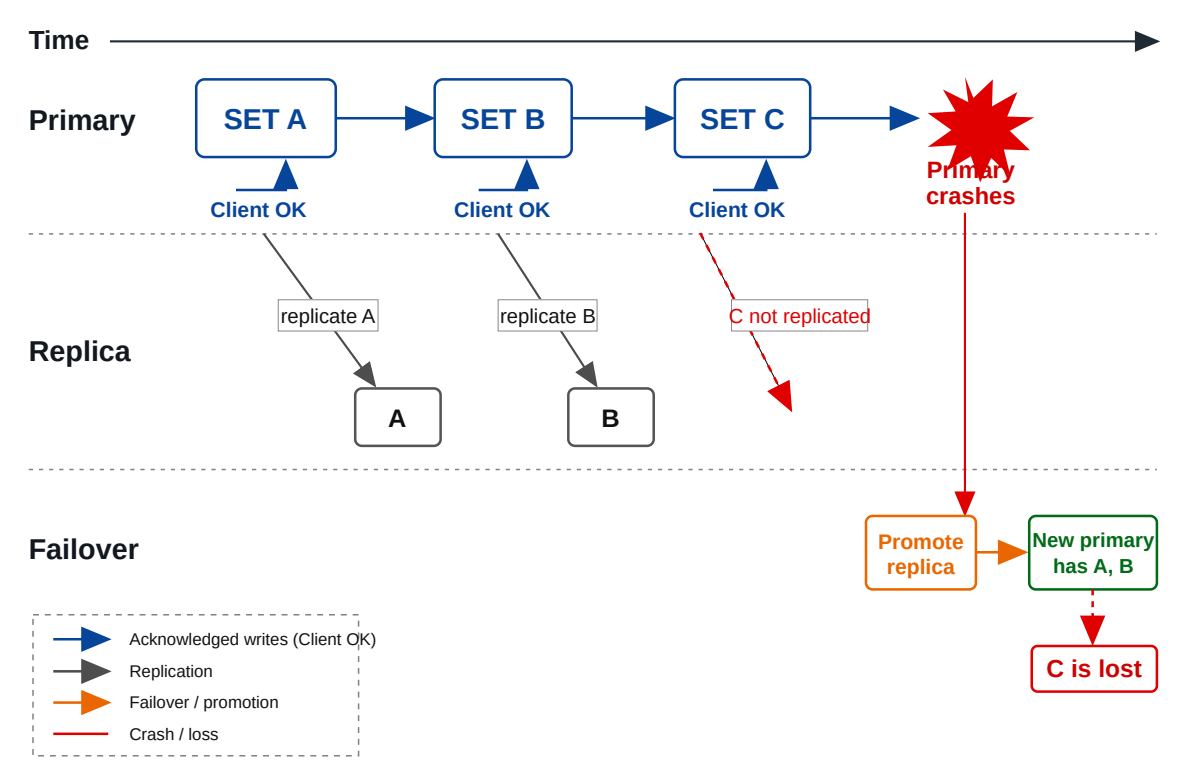
Step-by-step

1. `fork()` — child gets a snapshot of all data pages via COW (no copy yet)
2. **Child writes a new, compact AOF** — only the latest value of each key, no per-command history
3. **Parent continues serving** — new commands keep appending to the **old AOF** AND to an in-memory **rewrite buffer**
4. **Child finishes** — parent flushes the rewrite buffer into the new AOF
5. `rename(new, old)` — atomic on POSIX. The old AOF is unlinked; the new AOF takes its place

Worked Example: Async Failover Loss

Problem

The Redis **primary** acknowledged writes A, B, C. Replication is async. The primary crashes after C.



C is lost. The promoted replica only received A and B.

✋ Check: Does `appendfsync always` Eliminate the Loss Window?

You set `appendfsync always` (every command `fsync`'d locally) but replication remains async.

A hardware failure destroys the primary's disk. What happens to write C?

- A. C is safe — it was `fsync`'d
- B. C is lost — it never reached the replica
- C. It depends on whether Sentinel detected the failure fast enough

> **Answer: B.** Local durability survives a primary restart if the disk remains intact. It does not survive primary disk loss. The only copy of C was on that disk. `appendfsync` controls local durability; replication controls survival beyond one machine.

Common Mistakes with Redis

❌ **“Redis is unreliable”** → Redis is reliable for its designed workload. It is a cache / session / queue store, not a bank ledger.

❌ **“Setting `appendfsync always` makes Redis as safe as a database”** → Local fsync survives crash but not disk destruction. Without synchronous replication, acknowledged writes can still be lost on failover.

❌ **“Replication = durability”** → Async replication helps availability (a replica can take over), but acknowledged writes in the replication gap are lost. Only synchronous replication or quorum commit provides committed durability.

Redis Through the Storage-Design Lens

QUESTION	REDIS'S ANSWER
Truth first?	RAM — the in-memory data structures are the live state
When is `OK` honest?	Depends on `appendfsync`: memory, group flush, or per-write `fsync`
Read path?	Primary memory; replicas can serve reads but may lag
Failure path?	AOF/RDB help local recovery; async failover can lose acknowledged writes
Cleanup?	RDB snapshot, AOF rewrite, replica catch-up
Boundary?	One command is atomic on the event loop; wider transactions are limited

Unit B

etcd: Correctness in the Foreground Path

etcd: Design

etcd assumes some writes matter enough that **agreement and durability must happen before success is reported**.

Definition etcd is a distributed key-value store that provides linearizable reads, revisioned history, and reliable watch streams, built on the Raft consensus protocol.

When it matters Cluster metadata, distributed locks, leader election, configuration management — any workload where correctness matters more than raw throughput.

> **Contrast with Redis:** Where Redis says `OK` first and persists later, etcd persists and replicates before saying `OK`.

etcd: Why Kubernetes Needs It

Kubernetes needs **one place** to keep facts:

- This pod object exists
- This node is healthy
- This configuration changed at revision N

That workload is **mostly small metadata**, many reads, few writes — but those writes must be hard to dispute.

The write rate is low (control plane, not data plane), so etcd's higher latency is acceptable. The correctness requirement is high, so etcd's consensus cost is justified.

Worked Example: Tracing an etcd Write

Problem

A client sends `PUT /registry/pods/web running` to etcd. What happens before the client sees `OK`?

Step-by-step

```
Client ──► API Server ──► Leader proposes entry
                                  │
                                  ▼
                        ┌──── Raft Log ────┐
                        │  append + fsync  │  (leader)
                        │  replicate       │──► followers append + fsync
                        │  wait for quorum │
                        └──────────────────┘
                                  │ (majority have fsync'd)
                                  ▼
                        Apply to bbolt state machine
                                  │
                                  ▼
                        Return OK to client
```

etcd: WAL + bbolt Two-Layer Architecture

LAYER	PURPOSE	FORMAT
WAL	Crash recovery + Raft replication	Sequential append, one entry per Raft log entry
bbolt (B+ tree)	Efficient key-value reads + MVCC	On-disk B+ tree with bucket structure

Write path: WAL first (Raft commit) → then apply to bbolt

Read path: Directly from bbolt (no WAL scan needed)

Recovery: Replay WAL entries not yet applied to bbolt

> Same separation of concerns as the WAL principle from 10A: record intent first (WAL), apply to the main structure later (bbolt).

Concurrency model

etcd is written in Go: many **goroutines** (M:N over `epoll` at the kernel) handle RPCs, Raft messages, and watch streams in parallel. Unlike Redis's single-threaded loop, etcd parallelizes I/O freely — but **the bottleneck is consensus latency, not concurrency**, so the threading model is rarely what limits throughput.

etcd: MVCC and Watch

etcd doesn’t only store the latest value. It stores a **revisioned history window**:

```
$ etcdctl put /app/config "v1"     # → revision 5
$ etcdctl put /app/config "v2"     # → revision 8
$ etcdctl put /app/config "v3"     # → revision 12
```

Each mutation increments a **global revision counter**. Old values remain accessible until compacted.

This gives the control plane two things it needs:

- A **clean current state** — get the latest value
- A **consistent change stream** after revision R — reliable watch

etcd: Read and Failure Contract

etcd offers **two read modes** with different cost/freshness:

- **Linearizable read** (default): the leader confirms it is still the leader (a quick heartbeat round-trip) before answering. The reply reflects every write that committed before the read started.
- **Serializable read** (opt-in, e.g. `WithSerializable()`): the local node answers from its own bbolt without contacting peers. Cheap, but may be a few hundred ms behind the leader on a follower.

Why two modes?

Linearizable is correct-by-default. Serializable is the escape hatch when you can tolerate stale reads in exchange for lower latency and less load on the leader.

The cost paid on the write path (Raft commit + fsync) is what lets the read path be honest about freshness, and the failure path refuse to invent truth.

Worked Example: MVCC Revisions in Practice

Problem

How can you see the revision metadata for a key, and read an old value?

```
# Write some metadata
$ etcdctl put /registry/pods/web running
OK
$ etcdctl put /registry/pods/web terminating
OK
# Read with revision metadata
$ etcdctl get /registry/pods/web -w json | python3 -m json.tool
{
  "kvs": [{
    "key": "/registry/pods/web",
    "value": "terminating",
    "create_revision": 5,
    "mod_revision": 6,
    "version": 2
  }],
  "header": { "revision": 6 }
}
```

Worked Example: MVCC Revisions in Practice

```
# Read the old value at a past revision
$ etcdctl get /registry/pods/web --rev=5
/registry/pods/web
running
```

Key fields

`create_revision` = when the key was first created

`mod_revision` = last modification

`version` = how many times this key has been written

etcd Write Cost: Why Every Row Matters

COST ON THE WRITE PATH	REDIS	ETCD
Local durable log flush (`fsync`)	Optional	**Always**
Network round-trip before success	Usually no	**Yes**
Quorum coordination	No	**Yes**
Committed replica state before `OK`	Often no	**Yes**

Every row is a cost etcd **chose to pay**. The resulting write latency is not a weakness — it is the receipt for the guarantee.

> In Lab 10B, you will measure this directly: etcd write latency vs. read latency. The gap is the consensus cost.

etcd: Compaction and Defragmentation

MVCC revisions accumulate. Without cleanup: database grows, reads slow down, watch from old revisions gets expensive.

```
# See current DB size
$ etcdctl endpoint status --write-out=table

# Compact: discard all revisions before revision 1000
$ etcdctl compact 1000

# Defrag: reclaim freed space in the bbolt file
$ etcdctl defrag
```

Compaction removes old revision data (logical cleanup). **Defrag** reorganizes the B+ tree file (physical cleanup). Both are necessary — compaction alone leaves holes in the file.

> **Caution:** After `compact 1000`, a `watch --rev=500` will fail — that history is gone. This is the trade-off: space efficiency vs. history depth.

💬 Discussion

Your team is building a leaderboard service that updates scores **10,000 times per second**.

A colleague proposes using etcd. What's your response?

Think about:

- Write latency per operation (Raft round-trip + fsync)
- Total throughput ceiling of a 3-node etcd cluster
- Whether leaderboard scores need quorum-committed consistency
- What system would be a better fit?

✋ Check: etcd vs. Redis WAIT

Redis has a `WAIT` command: `SET key value` then `WAIT 1 0` (wait for at least 1 replica to acknowledge).

If you use `appendfsync always` + `WAIT 1 0` on every Redis write, does Redis's write path become equivalent to etcd's?

- A. Yes — both now fsync locally and wait for a replica
- B. No — Redis WAIT doesn't fsync on the replica, and there's no consensus log
- C. No — but the durability guarantee is equivalent

> **Answer: B.** Redis `WAIT` confirms the replica received the data, but the replica may not have fsync'd it. There is also no Raft log — so there's no formal consensus on ordering. Closer, but not equivalent. The gap: no durable replica, no total ordering, no safe leader election based on log state.

Common Mistakes with etcd

❌ **“etcd is slow”** → etcd is appropriately slow. The latency is the cost of the consistency guarantee.

❌ **“We should store everything in etcd because it’s consistent”** → etcd is designed for small, critical metadata. Storing large datasets or high-rate application data will overwhelm the Raft log and bbolt.

❌ **“Compaction is just garbage collection — do it whenever”** → Compaction destroys history. Any watch or time-travel read targeting a compacted revision will fail. You must plan compaction around your watch consumers’ checkpoint intervals.

etcd Through the Storage-Design Lens

QUESTION	ETCD'S ANSWER
Truth first?	Raft log entry, fsync'd on a quorum, then applied to bbolt
When is `OK` honest?	After the write is durably committed by a majority
Read path?	MVCC state in bbolt; linearizable reads confirm the current Raft view
Failure path?	No quorum → no writes; committed entries survive single-node failure
Cleanup?	Snapshot, compaction, defrag
Boundary?	Raft gives a total order of committed mutations

Unit C

LSM Engines: Append Now, Reorganize Later

LSM: The Scaling Problem It Solves

Suppose:

- The dataset is much larger than RAM
- Writes are frequent
- Random in-place updates are too expensive (random I/O on disk)

Neither Redis (everything in RAM) nor etcd (small metadata only) solves this.

A different strategy: **append a durable record quickly, keep recent state sorted in memory, flush to immutable on-disk files, merge those files in the background.**

That is the Log-Structured Merge-Tree (LSM) family. **Examples in production:** RocksDB (Meta), LevelDB (Google), Apache Cassandra (Apple/Netflix), BigTable / HBase (Google), TiKV / TiDB, CockroachDB (Pebble), ScyllaDB.

Worked Example: Life of an LSM Write

A client sends `PUT("user:1001", "Alice")` to an LSM engine. What happens?

Make the write durable immediately (WAL), keep it queryable (MemTable), then reorganize later (flush + compaction). Never do random I/O on the write path.

Step-by-step

```
Step 1: Client sends PUT("user:1001", "Alice")
            |
Step 2: Append to WAL (sequential write + fsync)   ← durability point
            |
Step 3: Insert into MemTable (sorted in-memory buffer, e.g. red-black tree)
            |
Step 4: When MemTable is full → flush to SSTable on disk (immutable sorted file)
```

Steps 2 and 3 are sequential / in-memory. No random disk I/O on the write path. The expensive reorganization (compaction) happens later, in the background.

Replay the WAL to rebuild any MemTable entries that weren't flushed. Same principle as 10A, same pattern as Redis AOF.

LSM Write Path in Pseudocode

```
def put(key, value):
    # Step 2: WAL for crash recovery
    wal.append(key, value)
    wal.fsync()                     # ← durability point
    # Step 3: MemTable for fast reads of recent writes
    memtable.insert(key, value)    # sorted in-memory structure
    # Step 4: flush when MemTable reaches size limit
    if memtable.size() ≥ THRESHOLD:
        frozen = memtable           # freeze current MemTable
        memtable = new_memtable()   # create a fresh one
        flush_to_sstable(frozen)    # write sorted file to disk
        wal.truncate()              # old WAL entries no longer needed
```

Three familiar patterns:

- WAL before data modification (10A principle)
- Immutable snapshots (like Redis fork + COW — freeze, don’t modify)
- Deferred cleanup (WAL truncation after flush)

SSTables: Why Immutability Is the Key Insight

When a MemTable fills up, it is **frozen** and flushed to disk as an **SSTable** (Sorted String Table) — the on-disk, immutable form of a MemTable's contents. From this point on, that data is no longer mutable in place.

An SSTable has three properties:

PROPERTY	WHY IT MATTERS
Sorted by key	Enables binary search and efficient range scans
Immutable once written	No partial-update risk — the file is complete or absent. Crash reasoning is simple
Self-contained	Each SSTable carries its own index and optional Bloom filter

> Immutability is the same principle as WAL: instead of updating files in place (torn-write risk), write a new file and delete the old one.

The Read Path: Where Amplification Lives

To read `GET("user:1001")`, the engine checks multiple locations:

```
1. MemTable (RAM)              → found? return
2. Immutable MemTable          → found? return   (if one is being flushed)
3. Level-0 SSTables            → check each (may overlap in key ranges)
4. Level-1 SSTables            → binary search within level (no overlap)
5. Level-2 SSTables            → binary search
   ...
```

Each level that doesn't contain the key is a **wasted read** — this is read amplification, the price LSM pays for its cheap write path.

Bloom filters eliminate most wasted reads: "not in this SSTable" → skip without touching disk.

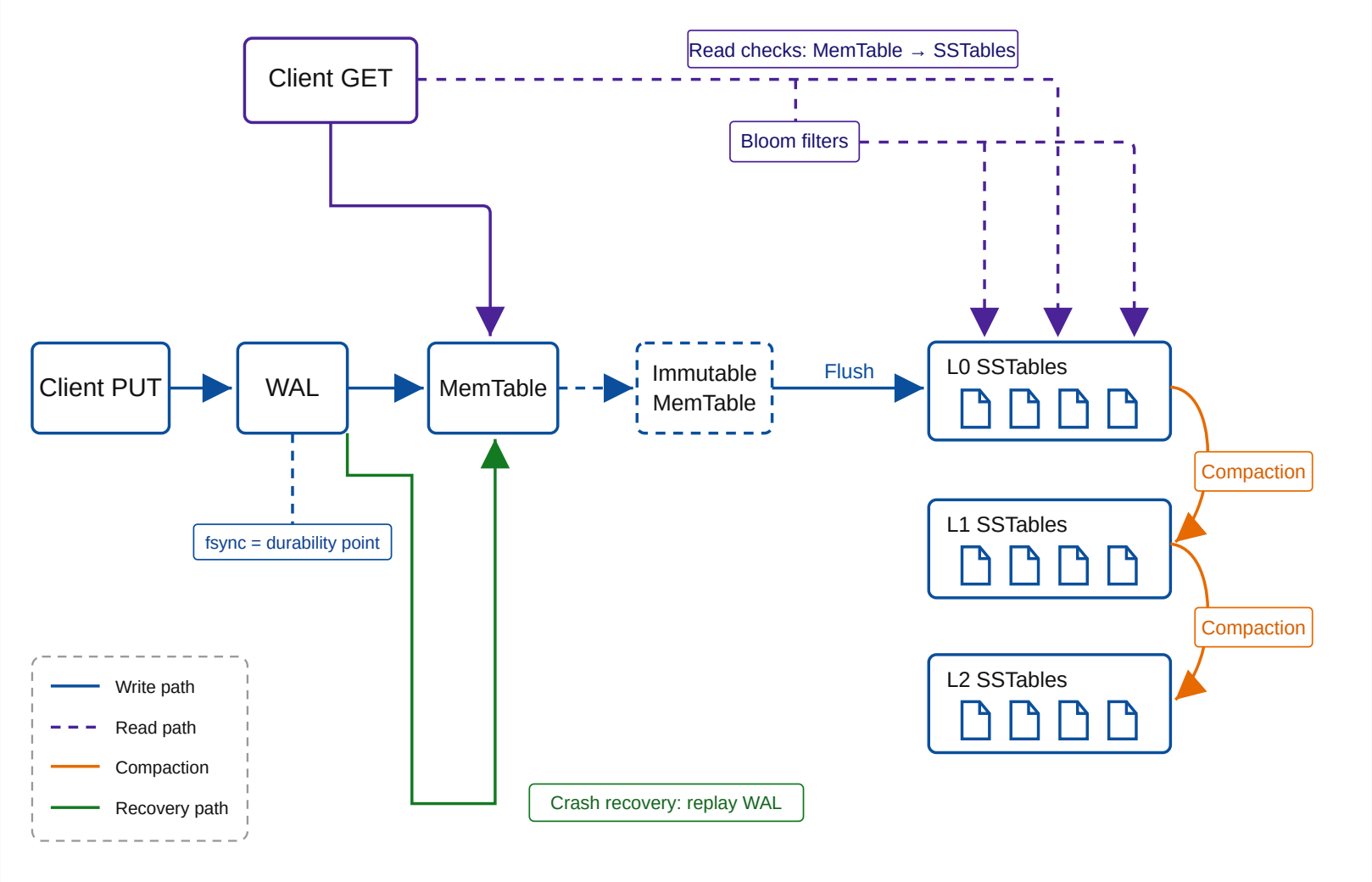
LSM: Failure Contract

The WAL and immutable files make crash recovery mechanical:

CRASH POINT	RECOVERY RULE
Before WAL `fsync`	Write is absent — never acknowledged
After WAL `fsync`, before flush	Replay WAL into a fresh MemTable
During SSTable flush	Partial file lacks a valid footer/checksum and was never registered in the manifest — delete it, replay WAL
During compaction	Keep old SSTables until the new set is atomically installed in the manifest

The design avoids in-place overwrite. An SSTable becomes visible only when the engine commits it to the **manifest** — until then, a half-written file on disk is just garbage to be deleted on restart. The data isn't lost: the WAL still has those records.

LSM: The Full Lifecycle



Three Amplification Costs: The LSM Triangle

AMPLIFICATION	PLAIN-LANGUAGE MEANING	EXAMPLE
Write	One logical write is rewritten multiple times during compaction	Your 1 KB write gets rewritten 5–10× across compaction passes
Read	One lookup consults multiple SSTables before finding the answer	GET checks MemTable + 3 levels before finding the key in L2
Space	Old and new versions coexist while merges are in progress	During compaction, both old and new SSTables exist on disk

These three form a **cost triangle** — tuning an LSM engine means shifting cost between them:

- More frequent compaction → lower read/space amp, higher write amp
- Less frequent compaction → lower write amp, higher read/space amp

Compaction: Not a Bug, Part of the Design

Without compaction, an LSM engine **degrades**:

- Reads touch too many files (read amplification grows unboundedly)
- Bloom filter effectiveness drops (too many SSTables to check)

Compaction is **continuous background reorganization**:

1. Select overlapping SSTables from adjacent levels
2. Merge-sort them into new, larger SSTables
3. Atomically replace old SSTables with new ones

> **Same design pattern everywhere:** Redis AOF rewrite, etcd compaction/defrag, LSM compaction. Fast appends now → deferred cleanup later. The mechanism differs; the principle is universal.

LSM in the Wild: Real-World Systems

SYSTEM	LSM ENGINE	USE CASE
RocksDB	(is the engine)	Embedded KV — used inside other systems
Apache Cassandra	Custom LSM	Wide-column distributed database
TiKV	RocksDB	Distributed transactional KV (powers TiDB)
CockroachDB	Pebble (Go)	Distributed SQL database
ScyllaDB	Custom (C++)	Cassandra-compatible, high-performance

RocksDB is the most important: it's the LSM engine inside many other systems. When you hear "TiKV uses RocksDB," TiKV handles distribution/consensus on top; RocksDB handles local sorted storage underneath.

✋ Check: Where's the Durability Point in LSM?

A client sends `PUT(key, value)` and receives `OK`. One second later, the machine loses power. The MemTable was NOT flushed to an SSTable yet.

Is the write lost?

- A. Yes — the MemTable was in RAM and is gone
- B. No — the WAL has the entry and recovery will replay it
- C. It depends on whether the SSTable flush had started

> **Answer: B.** The durability point is the WAL `fsync`, not the SSTable flush. The MemTable is a query-path optimization, not the source of durability. Recovery replays the WAL to reconstruct unflushed MemTable entries. Same as 10A: the log is the truth, not the data structure.

Common Mistakes with LSM Engines

❌ **“LSM engines just append forever — eventually they run out of disk”** → Compaction continuously merges and discards stale versions. Without it reads degrade, but it does actively reclaim space. The “append” part is only the write path; the lifecycle includes compaction.

❌ **“LSM is always better than B-tree for writes”** → LSM has cheaper sequential writes, but write amplification from compaction can be significant. For read-heavy workloads with modest write rates, a B-tree (like bbolt in etcd) may have lower total I/O cost.

❌ **“Bloom filters make reads as fast as in a B-tree”** → Bloom filters eliminate most negative lookups (key not present). But for keys that do exist across multiple levels, the engine still reads multiple SSTables. Read amplification is reduced, not eliminated.

LSM Through the Storage-Design Lens

QUESTION	LSM ENGINE'S ANSWER
Truth first?	WAL for recovery; MemTable for recent reads; SSTables after flush
When is `OK` honest?	After the WAL durable point (single-node)
Read path?	MemTable + SSTable levels; Bloom filters skip many misses
Failure path?	Replay WAL after crash; swap immutable files atomically
Cleanup?	Flush, multi-level compaction, tombstone cleanup
Boundary?	Engine-dependent; often key/batch within one shard or process

Scope note. A standalone LSM engine (RocksDB, LevelDB, Pebble) is a **single-node, embedded** library. By itself it does not provide quorum writes or linearizable reads — those are properties of the **distributed system that uses LSM as its local store**. Cassandra layers tunable quorum on top; TiKV and CockroachDB layer Raft on top. So when comparing with Redis or etcd, place LSM at a different layer of the stack.

Object Storage: A Different Problem

Object Storage: Brief Comparison

PROPERTY	REDIS / ETCD / LSM KV	OBJECT STORE (S3, GCS, MINIO)
Unit of update	Key or small record	Whole object (PUT replaces entire blob)
Write pattern	Frequent mutation	Upload or replace
Latency goal	µs – ms	10s – 100s of ms
Partial update	Yes (SET field, INCR)	No (re-upload entire object)
Best fit	Cache, metadata, hot data	Backups, media, logs, data lake, model weights

Core lens, broader checks

The Complete Write-Path Map

SYSTEM	TRUTH LIVES FIRST	WHEN `OK` IS HONEST	BACKGROUND WORK	BEST FIT
Redis	RAM	Depends on `appendfsync`	Snapshot, AOF rewrite, replica catch-up	Cache, session, queue
etcd	Quorum WAL + bbolt	After majority `fsync`	Snapshot, compaction, defrag	Cluster metadata
LSM KV	WAL + MemTable → SSTables	After WAL `fsync`	Flush, merge, compaction	Large write-heavy KV
Object store	Service-managed layer	Service-specific	Rebalance, erasure coding, lifecycle	Large durable blobs

The Full Storage-Design Lens

Use the write path first. Then add the follow-up checks.

DIMENSION	QUESTION	WHY IT MATTERS
Write path	When is `OK` honest?	durability and latency contract
Read path	Where do reads come from?	staleness, read-your-writes, amplification
Failure path	What happens after loss or partition?	safety vs. availability
Maintenance	What cleanup is deferred?	tail latency, space, operational cost
Boundary	What is atomic and ordered?	what applications can safely build

The Universal Pattern

All three systems share the same structural pattern, applied differently:

	REDIS	ETCD	LSM
Fast write mechanism	In-memory SET	WAL append	WAL append + MemTable insert
Deferred cleanup	AOF rewrite (fork+COW+rename)	Compaction + defrag	Multi-level compaction
Cleanup trigger	AOF size threshold	Revision count / DB size	Level size thresholds
Cleanup cost	Memory (COW)	I/O + history loss	I/O + write amplification

> **The meta-lesson:** Cheap writes always create deferred work. Systems differ in when and how they pay for that work.

Key Takeaways

1. Start with the write path. Trace where the write lands and when `OK` returns. That gives you the durability contract.

2. Then ask what the write path hides. Reads may be stale, failures may change behavior, and cleanup may appear later as tail latency.

3. Boundaries matter. A system may make one command atomic, one log totally ordered, or one object replaceable. Those are different promises.

4. Match the promise to the workload. etcd is wrong for a leaderboard. Redis is risky for cluster truth. LSM engines are not magic; they move cost into reads and compaction.

Lab Preparation

Lab 10A — Redis Durability and Recovery

What you'll do: benchmark three `appendfsync` modes, crash Redis mid-write, observe RDB fork/COW **What you should review first:**

- `appendfsync` as an acknowledgement contract
- AOF rewrite as `fork` + COW + atomic rename
- async failover loss window

Lab 10B — etcd as Durable Metadata Under Consensus

What you'll do: observe revisions/watch, measure read vs. write cost, trigger compaction/defrag **What you should review first:**

- Raft round-trip + quorum `fsync` on each write
- MVCC revisions and watch streams

- compaction destroying old history

One-Sentence Summary

Start with the write path, then check reads, failures, cleanup, and boundaries: Redis, etcd, and LSM engines pay in different places because they promise different things to different workloads.